



FINAL YEAR PROJECT REPORT

PHOTOGROUPE

A PHOTO SHARING AND BOOKING PLATFORM

RIJEN MAHARJAN (C30109220144)

8TH SEMESTER

**BACHELORS OF INFORMATION AND COMMUNICATION TECHNOLOGY
SCHOOL OF SCIENCE AND TECHNOLOGY**

ASIA e UNIVERSITY

July 2026

DECLARATION SHEET

I understand and agree that my project will become part of the permanent collection of the Virinchi College Library and Project Archive. My signature below authorizes the release of this project to individuals requesting access for academic and research purposes.

..... *JULY 2026*

RIJEN MAHARJAN

C30109220144

This Final Year Project, entitled "**PHOTOGROUPE**" was developed and presented by "**Rijen Maharjan**" in partial fulfilment of the requirements for the award of the Bachelor in Information and Communication Technology (BICT) degree during the 8th Semester, July, 2026.

APPROVED BY

..... JULY 2026

Chiranjibi Shrestha

Project Supervisor

..... JULY 2026

Binay Malla

Academic Coordinator

..... JULY 2026

Rastra Bhushan Khadka

Principal

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to everyone who contributed to the preparation of the Final Year Project titled "PhotoGroupe."

We would like to sincerely thank Mr. Chiranjibi Shrestha, our academic facilitator, for his support, constructive feedback, and guidance, which helped us refine our approach and ensure the proposal met academic standards.

Our sincere appreciation goes to Mr. Aakash Simkhada (CEO) for his inspiration, industry perspective, and motivation, which helped us understand real-world problem-solving and the practical application of digital solutions in Nepal.

We are also thankful to Mr. Binay Malla, our Academic Coordinator, for his academic guidance, support, and encouragement throughout the process, ensuring a smooth workflow for preparing the proposal.

We would also like to express our gratitude to Mr. Rastra Bhusan Khadka, our Principal, for his leadership and for fostering an environment conducive to academic growth and project development.

We are deeply grateful to Virinchi College, School of Science & Technology, under Asia e University, for providing the academic environment, resources, and opportunities necessary to prepare this proposal effectively.

We would also like to acknowledge all faculty members and staff of the Bachelor of Information & Communication Technology (BICT) program for their continuous support, knowledge sharing, and motivation throughout our academic journey.

Finally, we express our gratitude to our families and friends for their constant encouragement, motivation, and support, which helped us stay focused and committed while preparing this project proposal.

ABSTRACT

PhotoGroupe is a web-based image-sharing and photographer booking platform built to close a gap left by existing platforms like Pinterest, Instagram, Flickr, and Behance: none of them let a client discover, verify, negotiate with, and pay a professional photographer inside one integrated system. PhotoGroupe combines Pinterest-style image sharing (pins, categories, albums, likes, comments) with a full photographer booking workflow profile verification, availability, real-time chat negotiation, escrow-based payments via eSewa, and an event-request bidding module where photographers compete for client jobs.

The system is built with a Java Spring Boot backend and MySQL database on the server side, and a React frontend, with Cloudinary handling media storage and WebSocket/STOMP powering real-time chat and notifications. Core functionality booking creation, cancellation rules, escrow release, and the 12% platform commission calculation is verified through JUnit/Mockito unit tests and Spring Boot integration tests, while the full booking-to-payment lifecycle was manually walked through end-to-end across client and photographer roles. Performance testing showed response times in the tens of milliseconds for typical requests, well within the range that feels instant to users.

The result is a platform where photographers can turn portfolio visibility into actual bookings without leaving the site, and clients can find, vet, and hire verified professionals with payment protected by escrow a combination none of the reviewed competitor platforms currently offer.

TABLE OF CONTENTS

DECLARATION SHEET	i
APPROVED BY	ii
ACKNOWLEDGEMENTS	iii
ABSTRACT	iv
TABLE OF CONTENTS	v
CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement.....	1
1.3 Relevance to the IT Field	2
1.4 Objectives	3
1.4.1 General Objective.....	3
1.4.2 Specific Objectives.....	3
1.5 Scope	4
1.6 Limitations	5
1.7 Target Audience	5
CHAPTER 2: LITERATURE REVIEW	7
2.1 Introduction	7
2.2 Review of existing platform	7
2.2.1 Pinterest.....	7
2.2.2 Instagram	8
2.2.3 Behance	8
2.2.4 Flickr	9
2.3 Gaps in Existing System	10
2.4 Proposed System	10
2.5 Description of the Proposed Solution.....	11
2.6 Comparison of Existing and Proposed Systems.....	12

2.7 Conclusion	15
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN	16
3.1 Introduction	16
3.2 Requirement Analysis	16
3.2.1 Functional Requirements.....	16
3.2.2 Non-functional Requirements	18
3.3 Software Development Methodology	18
3.4 Feasibility Study	19
3.4.1 Technical Feasibility	19
3.4.2 Economic Feasibility	19
3.4.3 Operational Feasibility	19
3.4.4 Schedule Feasibility	19
3.5 Data Collection Techniques.....	20
3.6 SWOT / Risk Analysis	20
3.7 System Architecture	22
3.8 Database Design	23
3.8.1 Entity Relationship Diagram	23
3.8.2 Database Schema.....	23
3.9 Data Flow Diagram (Level 0, 1, 2)	25
3.10 UML Diagrams	27
3.10.1 Use Case Diagram	27
3.10.2 Class Diagram	28
3.10.3 Sequence Diagram.....	29
3.10.4 User Interface Design — Wireframes	30
CHAPTER 4: SYSTEM IMPLEMENTATION.....	33
4.1 Introduction	33
4.2 Technologies Used.....	33
4.3 Development Environment.....	35
4.4 Module-wise Implementation.....	35

4.5 Security Implementation	36
4.5.1 User Authentication.....	36
4.5.2 Authorization and Role Management.....	36
4.5.3 Password Hashing	36
4.5.4 Session Management.....	37
4.5.5 Input Validation.....	37
4.5.6 Data Protection and Security Measures.....	37
4.6 API Design and Implementation	37
4.7 Version Control and Project Management.....	38
4.7.1 Git/GitHub Workflow	38
4.7.2 Branching Strategy	39
4.7.3 Issue Tracking (GitHub).....	39
4.8 Resources Required	39
4.8.1 Software Requirements	39
4.8.2 Hardware Requirements	40
4.9 Deployment Details	40
4.9.1 Hosting Environment	40
4.9.2 Server Configuration	40
4.10 Project Timeline (Gantt Chart)	41
CHAPTER 5: TESTING AND EVALUATION	42
5.1 Testing Strategy	42
5.1.1 Unit Testing.....	42
5.1.2 Integration Testing	42
5.1.3 System Testing	42
5.2 Test Cases and Results	43
5.3 Performance Evaluation	44
5.3.1 Response Time	44
5.3.2 Scalability.....	44
5.3.3 Resource Utilization (CPU, Memory, Storage).....	44
5.4 Results and Discussion	45

5.5 Comparison with Objectives	45
5.6 Challenges Faced	45
CHAPTER 6: CONCLUSION AND FUTURE ENHANCEMENTS	47
6.1 Summary	47
6.2 Limitations	47
6.3 Future Enhancements	47
6.4 Conclusion.....	48
REFERENCES	49
APPENDICES	Error! Bookmark not defined.
Appendix A: Source Code.....	Error! Bookmark not defined.
Listing A.1: BookingController	Error! Bookmark not defined.
Listing A.2: PinController	Error! Bookmark not defined.
Appendix B: User Interface Screenshots	Error! Bookmark not defined.
Appendix C: Test Reports	Error! Bookmark not defined.
Table C.1: Unit Test Cases and Results	Error! Bookmark not defined.
Appendix D: GitHub Repository Structure / Commit History	Error!
Bookmark not defined.	
GitHub Repository Structure.....	Error! Bookmark not defined.
GitHub Commit History	Error! Bookmark not defined.
Table D.1: GitHub Commit History	Error! Bookmark not defined.

LIST OF TABLES

Table 2.6.1: Feature Comparison Matrix	12
Table 3.2.1: Functional Requirements	16
Table 3.2.2: Non-Functional Requirements	18
Table 3.4.1: Project Schedule and Deliverables.....	19
Table 3.6.1: SWOT Analysis Matrix.....	20
Table 3.6.2: Top Risks and Mitigation Strategies	21
Table 3.8.1: Database Schema Summary.....	23
Table 4.2.1: Technology Stack.....	33
Table 4.6.1: Sample API Endpoints	37
Table 4.8.1: Software Requirements	39
Table 4.8.2: Hardware Requirements.....	40
Table 5.2.1: Sample Test Cases and Results	43

LIST OF FIGURES

Figure 3.4.4.1: PhotoGroupe System Achitecture Diagram.....	22
Figure 3.8.1.1: Entity relationship diagram, core tables.....	23
Figure 3.8.2.1: Data Flow Diagram - Level 0 (Context Diagram).....	25
Figure 3.8.2.2: Data Flow Diagram - Level 1	25
Figure 3.8.2.3: Data Flow Diagram - Level 2 (Process Payments).....	26
Figure 3.10.1.1: Use case diagram. Dashed arrows denote «include» relationships.	27
Figure 3.10.2.1: Class Diagram.....	28
Figure 3.10.3.1: Sequence diagram for a Booking, Payment & Payout flow.	29
Figure 3.10.4.1: UI Wireframe – Home page.....	30
Figure 3.10.4.2: UI Wireframe – Workshop detail viewing and reserve seats page .	30
Figure 3.10.4.3: UI Wireframe – Photographer Portfolio	31
Figure 3.10.4.4: UI Wireframe - Booking Registration	31
Figure 3.10.4.5: UI Wireframe - Admin Dashboard Overview	32
Figure 3.10.4.6: UI Wireframe - Admin Transactions Panel	32
Figure 4.9.2.1: Project Timeline - Gantt Chart.....	41

LIST OF ABBREVIATIONS

Abbreviation Full Form		Abbreviation Full Form	
API	Application Programming Interface	MVP	Minimum Viable Product
CORS	Cross-Origin Resource Sharing	npm	Node Package Manager
CPU	Central Processing Unit	ORM	Object-Relational Mapping
CRUD	Create, Read, Update, Delete	OTP	One-Time Password
CSS	Cascading Style Sheets	RAM	Random Access Memory
DBMS	Database Management System	RBAC	Role-Based Access Control
FK	Foreign Key	REST	Representational State Transfer
GB	Gigabyte	SDK	Software Development Kit
HTTP	Hypertext Transfer Protocol	SMTP	Simple Mail Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure	STOMP	Simple Text Oriented Messaging Protocol
IDE	Integrated Development Environment	SWOT	Strengths, Weaknesses, Opportunities, Threats
IT	Information Technology	UAT	User Acceptance Testing
JDK	Java Development Kit	UI	User Interface
		UUID	Universally Unique Identifier

Abbreviation Full Form

JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token

Abbreviation Full Form

UML	Unified Modeling Language
URL	Uniform Resource Locator
UX	User Experience

CHAPTER 1: INTRODUCTION

1.1 Background

The rapid growth of digital technology has transformed the way people create, share, and consume visual content. Photography, once limited to personal albums and professional studios, has become an essential part of everyday communication through online platforms and social media. Millions of photographs are uploaded and shared every day, allowing users to express creativity, preserve memories, and communicate ideas with audiences around the world. As internet accessibility and smartphone usage continue to increase, the demand for platforms that enable efficient image sharing and content discovery has grown significantly.

Existing image-sharing platforms such as Pinterest, Instagram, Flickr, and Behance provide users with powerful tools to upload, organize, and discover visual content. These platforms have successfully created large communities where photographers and creative professionals can showcase their work and gain visibility. However, their primary focus remains social interaction, inspiration, and content engagement rather than providing complete professional services for photographers. Photographers who wish to earn income from their work often need to rely on multiple independent platforms for portfolio management, client communication, appointment scheduling, payment processing, and business promotion. This fragmented approach creates unnecessary complexity for both photographers and clients.

Professional photographers frequently face challenges in reaching potential clients directly, managing booking requests efficiently, and establishing credibility in a highly competitive digital marketplace. Similarly, users seeking photography services often struggle to identify verified professionals, compare portfolios, communicate effectively, and complete bookings without leaving the platform. The absence of an integrated ecosystem that combines image sharing with professional photography services represents a significant gap in existing digital platforms.

1.2 Problem Statement

Professional photographers and users face several challenges when using existing image-sharing and social media platforms for showcasing work and hiring photography services:

- **Lack of integrated booking systems** – Most image-sharing platforms focus on displaying and sharing visual content but do not provide built-in features for users to

book photographers directly, forcing them to rely on external communication channels.

- **Difficulty connecting photographers with potential clients** – Photographers often struggle to convert social media exposure into actual business opportunities because clients must search across multiple platforms to contact and hire them.
- **Limited professional portfolio management** – Existing platforms allow photographers to upload images but provide limited support for organizing portfolios by photography category, displaying service details, pricing, availability, or professional credentials.
- **Absence of photographer verification mechanisms** – Most platforms lack a structured verification process, making it difficult for users to distinguish verified professional photographers from unverified accounts, reducing trust and credibility.
- **Fragmented workflow across multiple platforms** – Photographers typically use separate applications for portfolio management, client communication, booking management, scheduling, and payments, resulting in inefficiency and increased administrative effort.
- **Limited monetization opportunities** – Current image-sharing platforms primarily emphasize social engagement through likes, comments, and followers, offering few direct opportunities for photographers to generate income from bookings, workshops, or professional services.
- **No centralized event request and bidding system** – Users seeking photographers for events such as weddings, receptions, or corporate functions cannot easily post photography requirements and receive competitive bids from multiple photographers within a single platform.
- **Lack of comprehensive management tools** – Existing platforms provide limited support for managing bookings, availability schedules, notifications, earnings, and analytics, making it difficult for photographers to efficiently operate and grow their businesses.

1.3 Relevance to the IT Field

PhotoGroupe is fundamentally an Information Technology (IT) project that applies modern software development practices to solve real-world challenges faced by photographers and clients. The project integrates various IT concepts, including web application development,

database management, system security, cloud-based media storage, and payment integration, to create a comprehensive platform for image sharing and photographer booking. Its development demonstrates the practical application of core IT principles in designing scalable, secure, and user-friendly software systems:

- **Web and Software Engineering** – The project requires designing and implementing a full-stack web application using modern frameworks (Java Spring Boot for the backend, React.js for the frontend), applying principles of RESTful API design, layered architecture, and maintainable software engineering practices.
- **Database Design and Management** – A relational database (MySQL) must be designed to securely and efficiently manage user accounts, subscriptions, transactions, and content, requiring careful schema design, normalization, and query optimization.
- **Payment System Integration** – Integrating third-party APIs such as eSewa handling secure transactions, API authentication, and real-time data exchange, which are core competencies in application integration and security.
- **Real-Time Systems** – Features like live donation alerts and OBS Studio integration require an understanding of real-time communication technologies such as WebSockets.

1.4 Objectives

1.4.1 General Objective

The general objective of this project is to design and develop **PhotoGroupe**, a web-based image-sharing and photographer booking platform that enables photographers to showcase their portfolios, connect with potential clients, and manage professional photography services through a secure and integrated digital system.

1.4.2 Specific Objectives

1. To design and develop a web-based platform that allows users to upload, share, search, and interact with images through features such as likes, comments, follows, and downloads.
2. To implement a professional photographer profile and portfolio management system that enables photographers to showcase their work and promote their photography services

3. To develop an integrated photographer booking system that allows users to browse photographer profiles, check availability, submit booking requests, and manage photography appointments.
4. To implement a photographer verification system and role-based access control to ensure secure platform management for Super Admins, Admins, Photographers, and Users.
5. To develop an event request and bidding module where users can create photography event requests and photographers can submit competitive bids based on client requirements.

1.5 Scope

The scope of **PhotoGroupe** covers the design, development, and testing of a web-based image-sharing and photographer booking platform. The system includes:

- **Image Sharing** – Upload, browse, search, like, comment, and download images.
- **Photographer Profiles** – Professional portfolios with an admin verification system.
- **Booking System** – Users can browse photographers, check availability, and book photography services.
- **Event Requests & Bidding** – Users can post event requests, and photographers can submit bids.
- **Admin Dashboard** – Manage users, verify photographers, monitor bookings, and generate reports.
- **Recommendations & Notifications** – Personalized image recommendations and booking-related notifications.

The project is limited to a **web-based platform** and does not include native mobile applications, advanced AI features, blockchain-based payments, or international multi-currency support in this phase.

1.6 Limitations

Although PhotoGroupe is designed to provide an integrated platform for image sharing and photographer booking, several limitations are recognized within the scope of this project:

- **Web-Based Platform Only** – The current implementation is developed as a responsive web application. Native Android and iOS applications are beyond the scope of this project due to time and resource constraints.
- **Limited Payment Gateway Integration** – While the system is designed to support secure online payments for bookings and event registrations, integration is limited to selected payment gateways. Support for additional national and international payment providers may be implemented in future versions.
- **Dependence on Internet Connectivity** – Since PhotoGroupe is an online platform, its performance depends on stable internet access. Users with slow or unreliable network connections may experience delays when uploading images, browsing content, or managing bookings.
- **Limited Real-World Testing** – As an academic project, system testing is conducted with a limited number of users and scenarios. The platform may require further testing and optimization before deployment in a large-scale production environment.
- **Scalability Constraints** – The current version is developed to demonstrate core functionalities. Handling a very large number of concurrent users, image uploads, and booking requests may require additional infrastructure, load balancing, and cloud-based storage solutions.
- **Advanced AI Features Not Included** – Advanced features such as AI-powered image recognition, automated image tagging, personalized recommendation algorithms, and fraud detection are outside the scope of the current implementation and are proposed as future enhancements.

1.7 Target Audience

PhotoGroupe is designed for users involved in photography and image sharing. The primary target audiences are:

- **Professional and Freelance Photographers** – Showcase portfolios, promote services, receive bookings, manage clients, and grow their photography business.

- **General Users (Clients)** – Browse images, discover photographers, and book photography services for various events.
- **Photography Enthusiasts** – Share images, interact with the community, and explore creative content.
- **Platform Administrators** – Verify photographers, moderate content, and manage the overall platform.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

This chapter reviews existing image-sharing and portfolio platforms to establish the foundation for the proposed PhotoGroupe system. It examines Pinterest, Instagram, Flickr, and Behance, focusing on their image-sharing features, portfolio management, and support for photographers. Although these platforms are effective for sharing and showcasing visual content, they lack integrated features such as photographer booking, verification, and event management. These limitations highlight the need for a unified platform that supports both image sharing and professional photography services. The chapter concludes by identifying the research gaps in existing systems and explaining how these findings influenced the design and development of PhotoGroupe.

2.2 Review of existing platform

2.2.1 Pinterest

2.2.1.1 Issues and Limitations

- Does not provide a built-in photographer booking system, requiring users to contact photographers through external channels (Pinterest Help, n.d.).
- Focuses primarily on image discovery and inspiration rather than professional photography services.
- Offers limited monetization opportunities for photographers.
- Does not include photographer verification or client management features.
- Users must rely on external platforms to contact or hire photographers (PicDrop, n.d.).

2.2.1.2 Key Lessons / Observations

- Visual discovery and categorized image collections improve user engagement (Pinterest Help, n.d.).
- Efficient search, tags, and recommendation features enhance content exploration.
- Professional service integration is necessary for photographers to convert visibility into business opportunities (Fiveable Content Team, 2025).

2.2.2 Instagram

2.2.2.1 Issues and Limitations

- Primarily designed for social networking rather than professional photography services (Instagram Help, n.d.).
- Booking and payment functionalities require third-party applications.
- Portfolio organization is limited for professional photographers.
- High competition and algorithm-driven visibility make client discovery difficult (Fstoppers, 2023).
- Does not provide photographer verification specifically for service providers.

2.2.2.2 Key Lessons / Observations

- Strong community engagement through likes, comments, stories, and messaging increases user interaction (Instagram Help, n.d.).
- High-quality visual presentation is important for attracting audiences.
- Direct communication features improve user engagement (Kumar & Patel, 2022).
- Integrating social interaction with professional booking services can provide greater value to photographers.

2.2.3 Behance

2.2.3.1 Issues and Limitations

- Designed mainly for showcasing creative portfolios rather than offering photography services.
- Does not support direct photographer booking or appointment management.
- Monetization depends largely on external client communication (Anderson, 2020).
- Limited features for managing photography-specific services and availability (Johnson & Miller, 2020).

- No integrated event management or bidding functionality.

2.2.3.2 Key Lessons / Observations

- Professional portfolio presentation strengthens personal branding. (Johnson & Miller, 2020)
- Community feedback and networking support creative growth (Kumar & Patel, 2022).
- Organized project showcases help photographers demonstrate expertise.
- Integrating portfolio management with client booking can transform creative exposure into business opportunities.

2.2.4 Flickr

2.2.4.1 Issues and Limitations

- Primarily focuses on image storage and sharing rather than client acquisition (Williams, 2026).
- Does not provide an integrated booking or scheduling system for photographers (Williams, 2026).
- Monetization opportunities are limited and mainly depend on external business channels (Wolinski, 2025).
- Lacks photographer verification and business management features (Williams, 2026).
- Client communication and booking management require external platforms (PicDrop, n.d.).

2.2.4.2 Key Lessons / Observations

- High-resolution image storage and portfolio organization are valuable for professional photographers (Wolinski, 2025).
- Image categorization and tagging improve content organization and discoverability (Williams, 2026).
- Usage statistics help photographers understand audience engagement (Williams, 2026).
- Integrating portfolio management with booking services would create a more complete professional platform (PicDrop, n.d.).

2.3 Gaps in Existing System

Based on the feature comparison and analysis of existing global platforms, the following gaps have been identified in the Nepali market:

- **Lack of Integrated Booking System:** Existing platforms focus on image sharing and social interaction but do not provide a built-in system for users to directly book photographers.
- **Limited Monetization Opportunities:** Photographers rely on external platforms or manual communication to acquire clients and generate income, resulting in fragmented workflows.
- **Absence of Photographer Verification:** Most platforms lack a structured verification process, making it difficult for users to identify trustworthy and professional photographers.
- **No Centralized Portfolio and Client Management:** Although images can be uploaded and organized, existing platforms do not integrate portfolio management with booking, scheduling, and client communication.
- **Limited Event Management and Bidding Features:** Current platforms do not allow users to post photography event requests or enable photographers to submit competitive bids for projects.
- **Lack of Availability and Scheduling Management:** Photographers cannot efficiently manage their availability, booking calendars, or time slots within a single platform.
- **Limited Analytics for Professional Growth:** Existing platforms provide basic engagement statistics but lack comprehensive insights into bookings, earnings, client inquiries, and business performance.

2.4 Proposed System

To overcome the limitations of existing image-sharing platforms, PhotoGroupe is proposed as a web-based platform that combines image sharing, professional portfolio management, and photographer booking within a single system. The platform enables photographers to showcase their work, receive bookings, manage availability, and participate in event bidding, while allowing users to browse images, discover photographers, and book photography services.

PhotoGroupe is developed using React.js for the frontend, Spring Boot for the backend, MySQL for database management, and Cloudinary for image storage. The system also implements JWT authentication and role-based access control to ensure secure and efficient platform management.

2.5 Description of the Proposed Solution

PhotoGroupe was built around the following core components:

- **Integrated Portfolio and Booking System** – Photographers can showcase categorized portfolios (weddings, events, portraits) while users book them directly through the platform, eliminating the need to switch between separate discovery and service-negotiation tools.
- **Role-Based Access Control** – Four distinct roles Super Admin, Admin, Photographer, and User each receive scoped permissions, from full platform oversight down to browsing and booking, ensuring accountability and appropriate access at every level.
- **Photographer Verification Workflow** – Admins review submitted credentials and portfolio samples before granting a verified badge, giving clients a trust signal that's largely absent from general-purpose photo-sharing platforms.
- **Event Request and Bidding Module** – Users can post structured event requests (Mehendi, Reception, Wedding, etc.) with dynamically updating pricing scope, and photographers submit competitive bids a marketplace mechanic not found in Pinterest, Instagram, or Flickr.
- **Availability and Booking Management** – A time-slot-based calendar lets photographers manage their schedules, with confirmed bookings automatically blocking selected slots to prevent double-booking.
- **Localized Payment Integration** – Direct integration with eSewa (and planned Khalti support) allows secure, encrypted transactions for both photographer bookings and workshop registrations, processed entirely within the platform.
- **Workshop and Event Hosting** – Photographers can create and manage their own workshops or events, complete with participant registration and payment collection, opening an additional revenue stream beyond one-off bookings.
- **Image Sharing and Discovery** – Users can upload, categorize, search, like, comment

on, and download images, preserving the social-discovery experience users expect from platforms like Pinterest while layering professional services on top.

- **Analytics and Reporting Dashboards** – Photographers, admins, and super admins each get tailored dashboards surfacing bookings, earnings, engagement metrics, and platform activity to support data-driven decisions.
- **Notification System** – Automated in-platform notifications keep users and photographers informed of booking confirmations, bid responses, workshop updates, and other time-sensitive activity.

2.6 Comparison of Existing and Proposed Systems

Table 2.6.1: Feature Comparison Matrix

Feature / Aspect	Pinterest	Instagram	Flickr	Behance	PhotoGrope (Proposed)
Core Focus	Content discovery & inspiration	Social networking & content sharing	Image storage & archival sharing	Creative portfolio showcasing	Image sharing + professional photography services
Photographer Booking	Not available	Requires third-party apps	Not available	Not available	Built-in booking system with availability management
Photographer Verification	Not available	Not available	Not available	Not available	Admin-approved verification workflow with badges

Client-Photographer Matching	Basic search/tags only	Algorithm-driven feed, no service matching	Tag-based search only	Portfolio browsing only	Filtered search by style, location, availability, budget
Event/Workshop Management	Not available	Not available	Not available	Not available	Photographer-created events & workshops with registration
Bidding System	Not available	Not available	Not available	Not available	Event request & bidding module (users post requests, photographers bid)
Monetization for Photographers	Limited/indirect	Requires brand deals or external tools	Limited, mainly via Pro subscription	Depends on external client contact	Booking commissions, premium features, promoted listings
Payment Integration	Not available	Not available	Not available	Not available	eSewa / Khalti secure payment gateway
Role-Based Access Control	Not applicable	Not applicable	Not applicable	Not applicable	Super Admin, Admin,

					Photographer, User roles
Portfolio Organization	Board-based collections	Grid feed, limited categorization	Albums, tags, statistics	Project-based showcases	Structured categories (weddings, events, portraits)
Analytics/Dashboard	Basic pin stats	Basic post insights	View/download statistics	Basic project views	Dashboard with bookings, earnings, engagement analytics
Notifications	Basic engagement alerts	Likes/comments/DMs	Basic activity alerts	Basic comment alerts	Booking confirmations, workshop updates, interaction alerts
Trust Mechanism	None	None	None	None	Verified professional badges + admin moderation
Centralization	Single-purpose (discovery)	Single-purpose (social)	Single-purpose (storage)	Single-purpose (showcase)	Unified: sharing + portfolio + booking + payment in one platform

2.7 Conclusion

PhotoGroupe addresses the limitations of existing image-sharing platforms by integrating image sharing, professional portfolio management, and photographer booking into a single platform. It enables photographers to showcase their work, connect with clients, and manage bookings efficiently, while providing users with a secure and convenient way to discover and hire photographers.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

PhotoGroupe is a web-based image-sharing and photographer booking platform designed to connect photographers with clients through a unified digital system. The platform is developed using React.js for the frontend, Spring Boot for the backend, MySQL for database management, and Cloudinary for image storage. It provides features such as image sharing, portfolio management, photographer verification, booking management, event requests, and role-based access control. This chapter presents the system analysis and design, including requirement analysis, feasibility study, system architecture, functional and non-functional requirements, UML diagrams, database design, and the overall design of the proposed system.

3.2 Requirement Analysis

The requirements for PhotoGroupe were identified through the analysis of existing image-sharing platforms, including Pinterest, Instagram, Flickr, and Behance, and by examining the challenges faced by photographers and clients. The analysis revealed the need for a platform that not only supports image sharing but also provides professional portfolio management, photographer verification, and an integrated booking system. These findings were used to define the functional and non-functional requirements of PhotoGroupe, ensuring that the platform effectively meets the needs of photographers, freelance professionals, and users within a single, centralized system.

3.2.1 Functional Requirements

Table 3.2.1: Functional Requirements

Module	Key Requirements
Authentication	Register/login with email/password or Google OAuth2; JWT access + refresh token flow; password reset; role-based accounts — USER, PHOTOGRAPHER, ADMIN, SUPER_ADMIN.
User Profile	Profile view/edit (bio, profile picture, cover image); follow/unfollow and block/unblock system; mutual-followers and follow-stat display; per-viewer-per-day profile view tracking.
Photographer Profile	Photographer verification workflow (admin-approved); custom free-text specializations; portfolio display separate from general pin feed; photographer ranking via Bayesian weighted rating (IMDb-style).
Image Sharing	Pin CRUD with soft delete; category assignment (by id/slug/name); tagging;

Pins	like, comment (with nested replies and comment-likes), save/bookmark, and share; weighted/personalized feed with category-preference sampling; related-pins and top-pins by engagement; view/download/share tracking.
Albums	Album CRUD with visibility (Public/Private); cover image upload; add pins to album from existing pins or direct upload; remove pin from album; zip download of full album via ZipOutputStream.
Categories	Category CRUD (admin-managed) with cover image, slug, and description; category browsing and keyword search; category-based pin filtering.
Booking System	Users browse photographers and submit booking requests; photographer accept/reject; booking status flow (PENDING → CONFIRMED → COMPLETED/CANCELLED/REJECTED/NO_SHOW); booking penalty enforcement; escrow-style payment hold/release on completion.
Event Request & Bidding	Users post structured event requests (Mehendi, Reception, Wedding, etc.); dynamic pricing scope by selected services; photographers submit competitive bids; user reviews and selects a bid.
Workshops	Photographer-created workshops with seat capacity; atomic seat reservation to prevent race conditions; participant registration and management; workshop-specific payment flow.
Payments	eSewa EPay v2 integration for bookings and workshop registrations; dedicated Payment records with sourceType routing (booking vs. workshop); Payout entity with 12% platform commission; signature verification and duplicate-increment safeguards on seat/payment confirmation.
Real-Time Chat	WebSocket (STOMP/SockJS) messaging; unread-message badge; in-chat package/booking proposal cards (PackageMessageCard) supporting PREMIUM accept/decline and CUSTOM counter-offer negotiation.
Notifications	Automated notifications for likes, comments, replies, follows, bookings, bids, workshop updates, and admin actions; notification bell with polling.
Admin Panel	Report management (pending/resolved); user, booking, event, and workshop moderation; banner ad and photographer-boost monetization tools with scheduled expiry; booking-trend analytics dashboard.
Monetization	Banner ads and photographer profile "boosts" with configurable AdPricingConfig; automatic expiry scheduling.
Sharing	ShareModal supporting multiple content types (pins, albums, workshops) beyond the original pin-only scope.
Theming	Light/dark mode via ThemeContext with localStorage persistence, applied across layout, sidebar, and portfolio components.

3.2.2 Non-functional Requirements

Table 3.2.2: Non-Functional Requirements

Category	Requirement
Security	JWT-protected APIs; hashed passwords; role-based access control (USER/PHOTOGRAPHER/ADMIN/SUPER_ADMIN); OAuth2 login; restricted CORS.
Performance	Paginated feeds/listings; N+1 query fixes via JPQL projections; images served via Cloudinary CDN.
Usability	Responsive UI with light/dark theme; consistent design system across all views.
Reliability	Payment status verified via eSewa callbacks only; atomic seat reservation prevents double-booking.
Scalability	Stateless JWT/REST API scales horizontally; single MySQL instance is the current bottleneck.
Maintainability	Interface-based service layer; Lombok boilerplate reduction; env-based config via dotenv.
Availability	WebSocket/STOMP for real-time chat/notifications; scheduled jobs for ad/boost expiry.
Portability	Cross-platform Java/Spring Boot backend; cloud-based MySQL and Cloudinary.

3.3 Software Development Methodology

PhotoGroupe was built using an Agile, iterative-incremental methodology. As a solo-developer project spanning both a React frontend and Spring Boot backend with evolving scope, requirements could not be frozen upfront each module (auth → pins/categories → albums → booking → bidding → workshops → chat → payments → admin) was designed, built, and tested end-to-end as its own short cycle, producing a working increment before the next began. This kept risk low and allowed later features (dark theme, banner ads/boosts, booking penalties, package negotiation) to layer cleanly onto a stable core (Beck et al., 2001).

3.4 Feasibility Study

3.4.1 Technical Feasibility

The stack (Spring Boot 4.0.4/Java 17, React, MySQL) is mainstream, well-documented, and matches standard curriculum skills. All third-party services (Cloudinary, Google OAuth2, eSewa) offer free sandbox/developer tiers, so the system was fully buildable and testable without paid infrastructure. Conclusion: technically feasible.

3.4.2 Economic Feasibility

Development cost was limited to developer time, since every tool used has a free tier. The system currently runs in a local development environment only and has not yet been deployed to a production host. Anticipated production costs VPS hosting, a domain name, and any Cloudinary usage beyond the free quota remain modest, and the platform's own 12% booking/workshop commission would be sufficient to cover these once deployed and generating transaction volume. Conclusion: economically justified.

3.4.3 Operational Feasibility

eSewa is already a widely adopted wallet among Nepali users, so no new payment behavior is required from photographers or clients. The main operational constraint is that photographer verification and booking-penalty enforcement are reviewed manually by an admin, since there is no automated identity-verification service integrated acceptable at current scale, but the clearest point needing automation before scaling up.

3.4.4 Schedule Feasibility

Table 3.4.1: Project Schedule and Deliverables

Phase	Semester	Deliverable
Proposal & analysis	7th	Scope, competitor review (Pinterest, Instagram, Flickr, Behance)
System design	7th	ER model, API design, wireframes
Core build (auth, pins, categories)	7th	JWT auth, pin CRUD, category browsing, masonry feed
Booking, bidding,	8th	Booking flow, event request/bidding, workshop

workshops		registration via eSewa
Chat, admin panel, monetization	8th	WebSocket Real-time chat, admin report/booking management, banner ads/boosts
Testing & deployment	8th	Manual/unit testing, hosting
Report writing	8th	Final report, defense prep

3.5 Data Collection Techniques

- **Competitor analysis:** studied Pinterest, Instagram, Flickr, and Behance to establish the baseline feature set (image sharing, portfolio showcasing, social engagement) and identify gaps in professional service integration.
- **Market-gap observation:** confirmed that existing image-sharing platforms lack built-in booking, verification, and event-bidding systems, forcing photographers to rely on external channels to convert visibility into paid work.
- **Gateway documentation review:** eSewa EPay v2 API documentation studied directly, which shaped the booking/workshop payment flow, signature verification, and seat-reservation confirmation logic.
- **Iterative dogfooding:** each module tested end-to-end during development, with critical flows manually verified through the running application.

3.6 SWOT / Risk Analysis

Table 3.6.1: SWOT Analysis Matrix

	Helpful	Harmful
Internal	Strengths: Image sharing, booking, bidding, and workshops in one platform; album zip download and share; real-time chat negotiation; native eSewa payments.	Weaknesses: Manual verification; single-instance database; solo-developer bandwidth; not yet deployed.
External	Opportunities: No competitor offers integrated photographer booking; room to	Threats: Gateway API/policy changes; regulatory shifts on digital payment

	add Khalti.	commissions; eSewa API changes; competitors adding booking features.
--	-------------	--

Top Risks

Table 3.6.2: Top Risks and Mitigation Strategies

Risk	Impact	Mitigation
Gateway downtime/API change	High	Payment logic isolated in its own service layer
Seat Booking race condition	Medium	Atomic SQL update prevents double booking
Single-DB data loss	High	Regular backups; migrate to managed DB before scaling
Unverified Photographer accounts	Medium	Admin-approved verification workflow

3.7 System Architecture

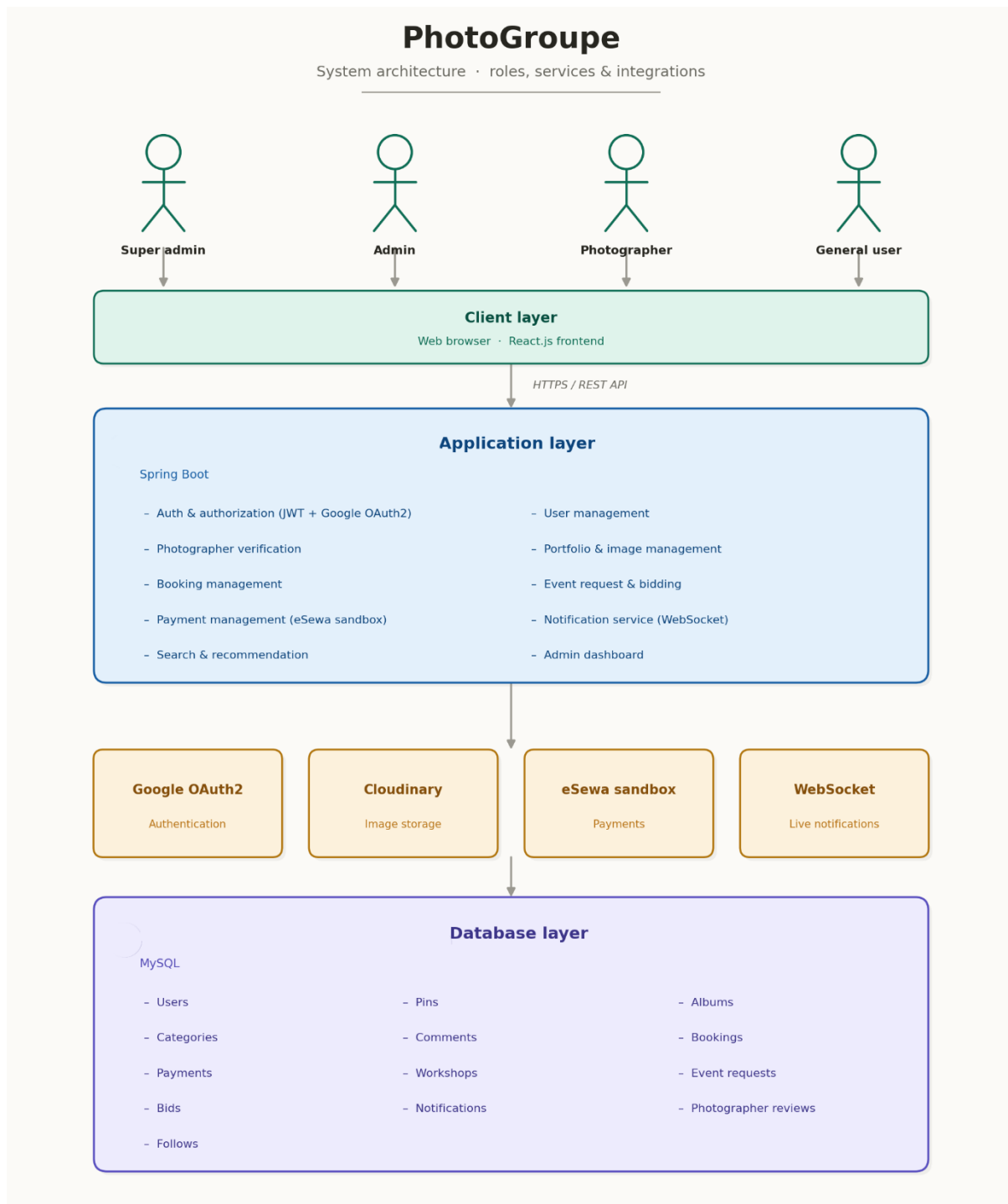


Figure 3.4.4.1: PhotoGroupe System Achitecture Diagram

3.8 Database Design

3.8.1 Entity Relationship Diagram

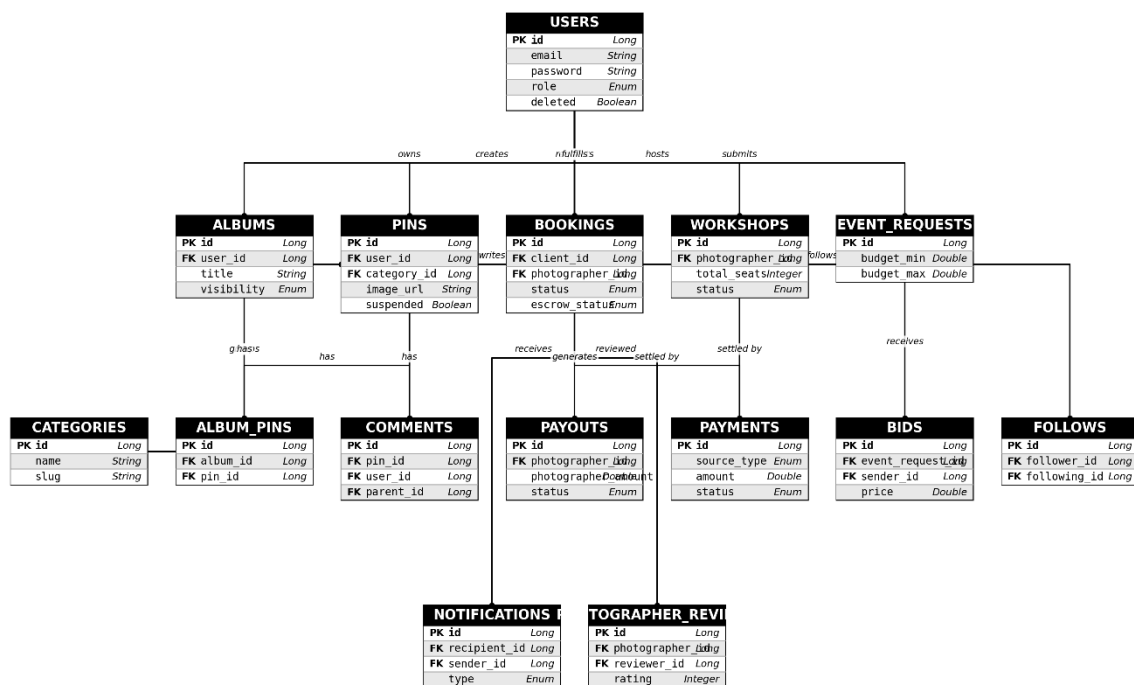


Figure 3.8.1.1: Entity relationship diagram, core tables

3.8.2 Database Schema

Table 3.8.1: Database Schema Summary

Table	Purpose	Key Columns
users	Account record for every user, photographer, admin, and super admin.	id, email, password, role, deleted
pins	Uploaded image posts, the core shareable content unit.	id, user_id (FK), category_id (FK), image_url, suspended
categories	Pin categories used for browsing and filtering.	id, name, slug, cover_image
comments	Comments on pins, supports nested replies via parent_id.	id, pin_id (FK), user_id (FK), parent_id (FK, self)
albums	User-owned pin collections with visibility control.	id, user_id (FK), title, visibility

album_pins	Junction table linking pins to albums (many-to-many).	id, album_id (FK), pin_id (FK)
bookings	Core booking record between a client and a photographer.	id, client_id (FK), photographer_id (FK), status, escrow_status
booking_packages	Negotiated package/pricing tied to a booking.	id, booking_id (FK), price, package_type, status
event_requests	Client-posted event requests open for photographer bidding.	id, budget_min, budget_max, event_type, status
bids	Photographer bids submitted against an event request.	id, event_request_id (FK), sender_id (FK), price, status
workshops	Photographer-hosted workshops with seat capacity.	id, photographer_id (FK), total_seats, seats_booked, status
workshop_participants	Registration record for a user in a workshop.	id, workshop_id (FK), participant_id (FK), status
payments	Unified payment log routed by source_type (booking or workshop).	id, booking_id (FK), source_type, amount, status
payouts	Photographer payout record with platform commission applied.	id, photographer_id (FK), booking_id (FK), photographer_amount, status
notifications	In-app notifications for likes, bookings, bids, and system events.	id, recipient_id (FK), sender_id (FK), type, is_read
follows	Follow relationships between users.	id, follower_id (FK), following_id (FK)
reports	User-submitted moderation reports reviewed by admins.	id, reported_user_id (FK), reporter_id (FK), reason, status
photographer_specializations	Free-text specialization tags for a photographer profile.	id, photographer_id (FK), custom_type, category_id (FK)

3.9 Data Flow Diagram (Level 0, 1, 2)

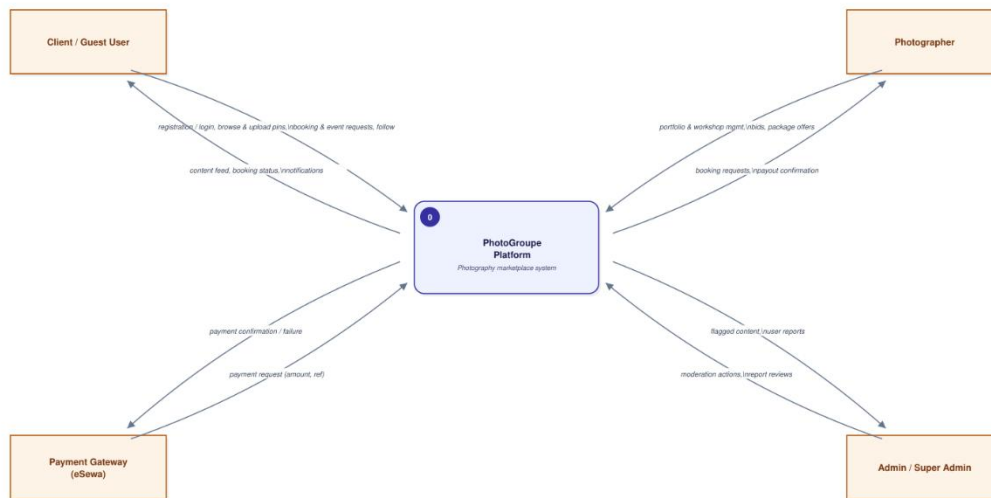


Figure 3.8.2.1: Data Flow Diagram - Level 0 (Context Diagram)

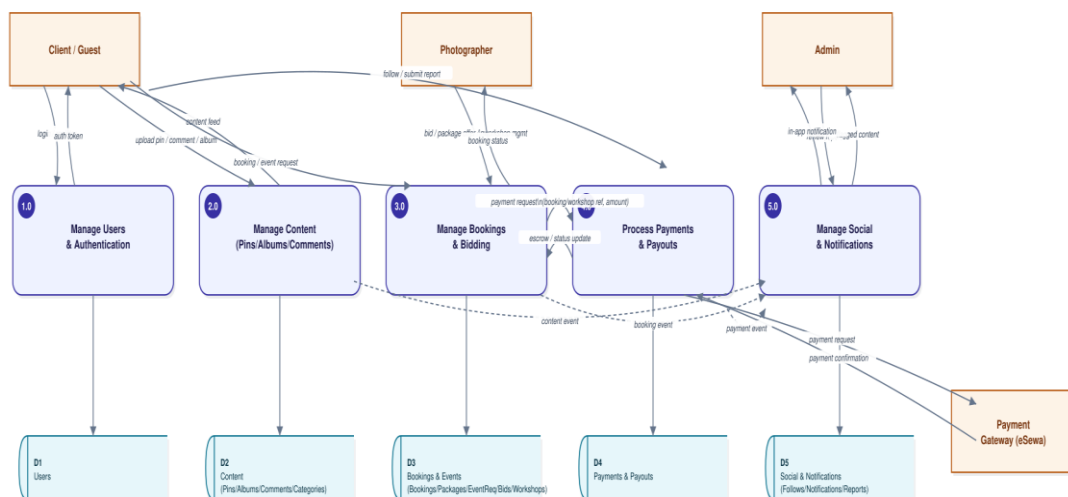


Figure 3.8.2.2: Data Flow Diagram - Level 1

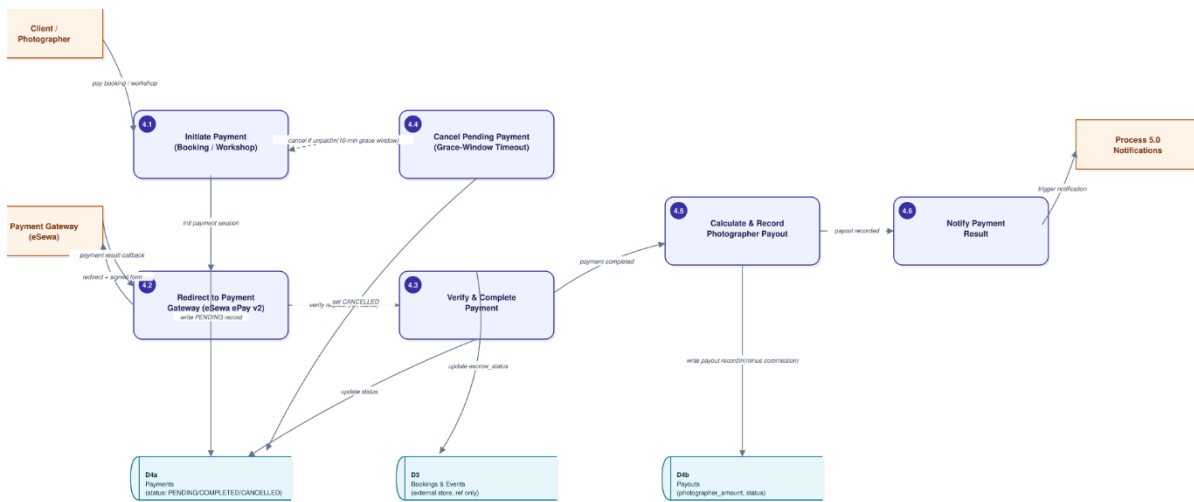


Figure 3.8.2.3: Data Flow Diagram - Level 2 (Process Payments)

3.10 UML Diagrams

3.10.1 Use Case Diagram

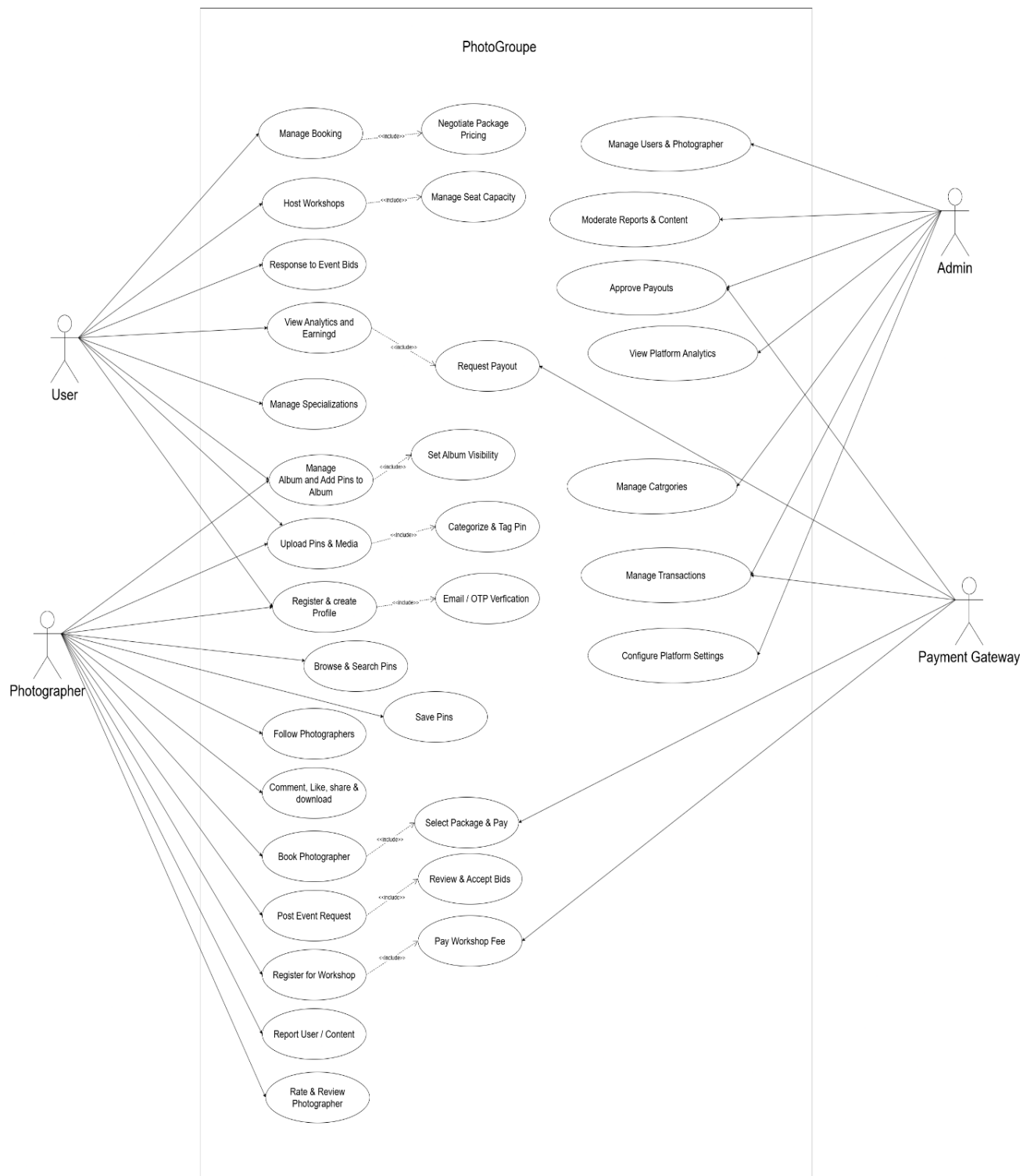


Figure 3.10.1.1: Use case diagram. Dashed arrows denote «include» relationships.

3.10.2 Class Diagram

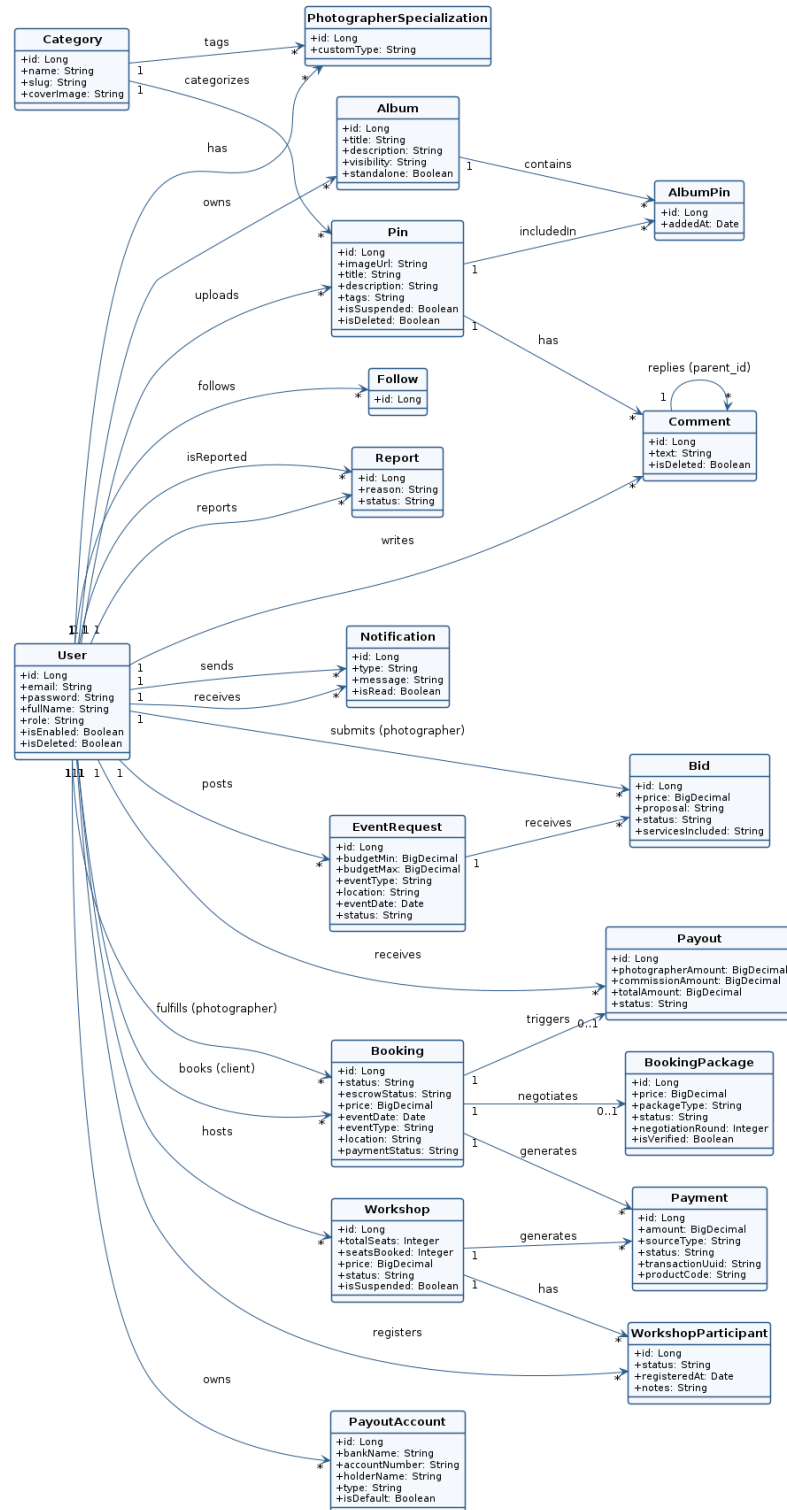


Figure 3.10.2.1: Class Diagram

3.10.3 Sequence Diagram

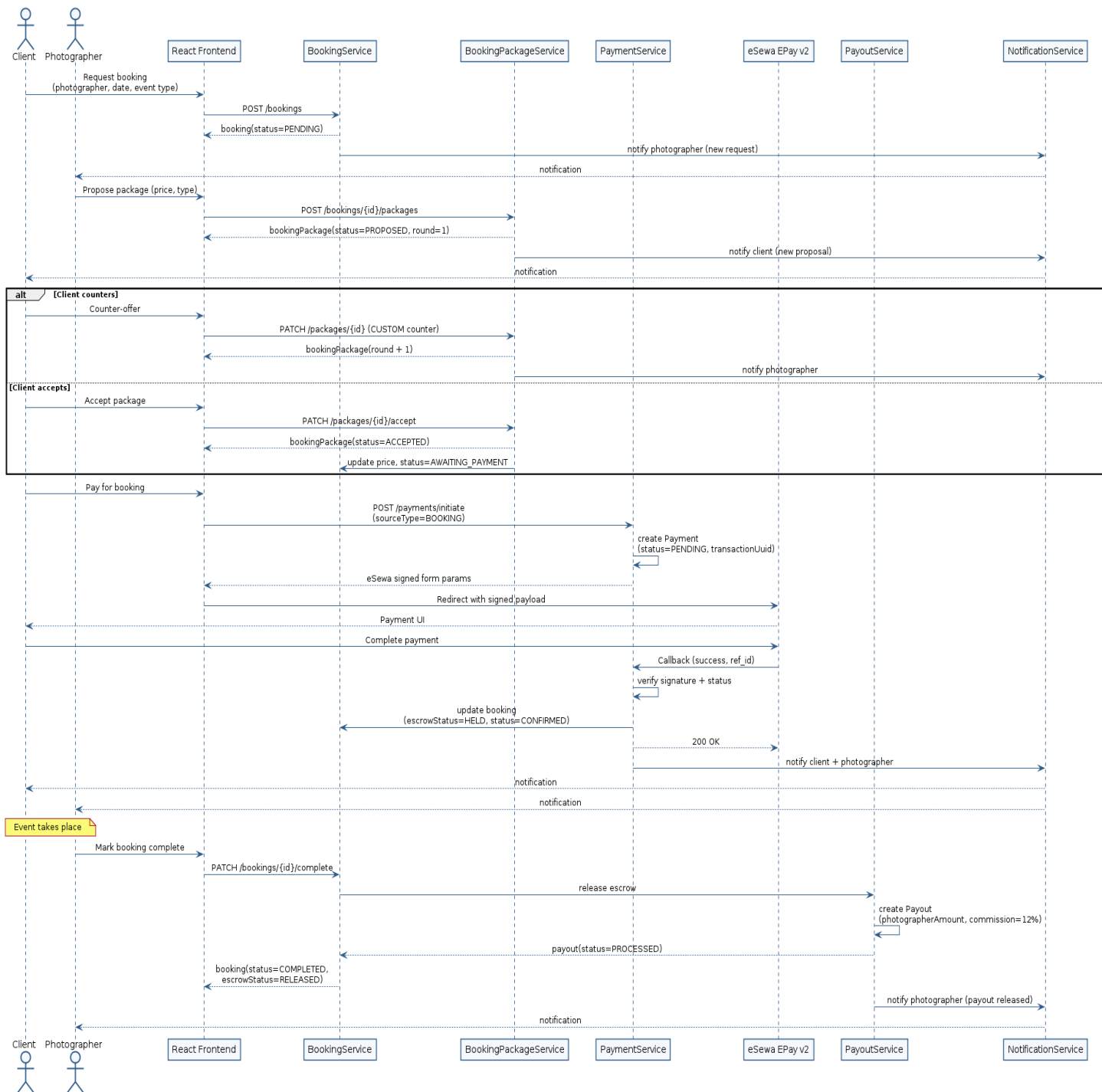


Figure 3.10.3.1: Sequence diagram for a Booking, Payment & Payout flow.

3.10.4 User Interface Design — Wireframes

Low-fidelity wireframes: Home page

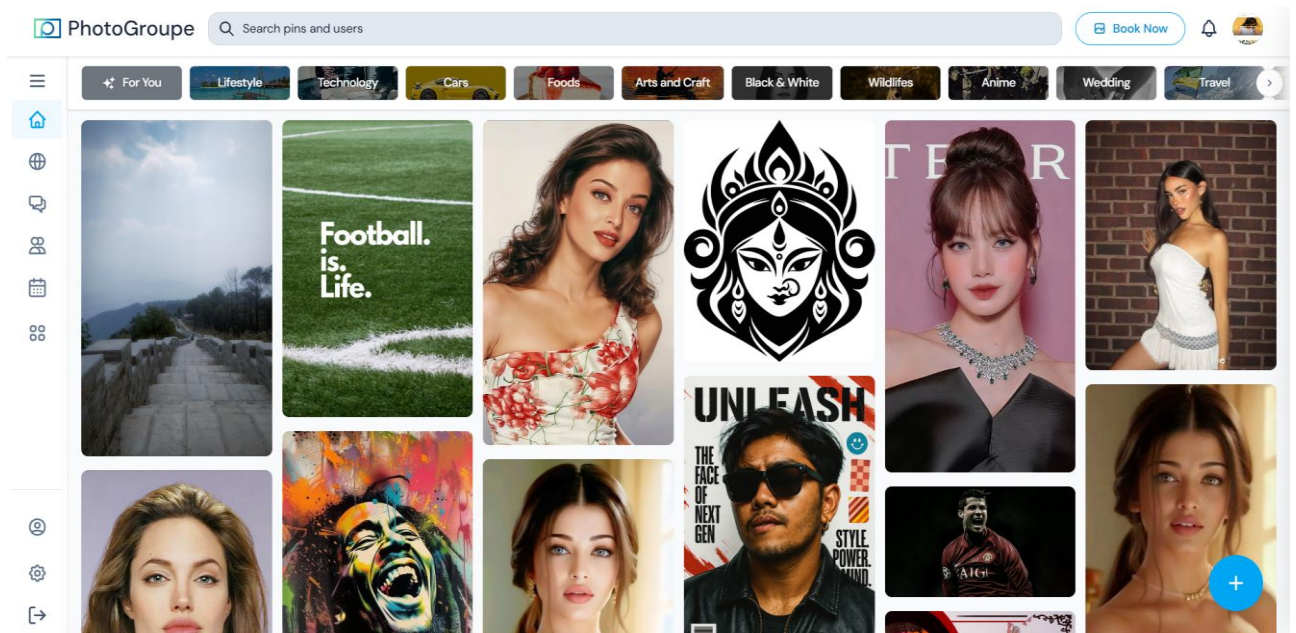


Figure 3.10.4.1: UI Wireframe – Home page

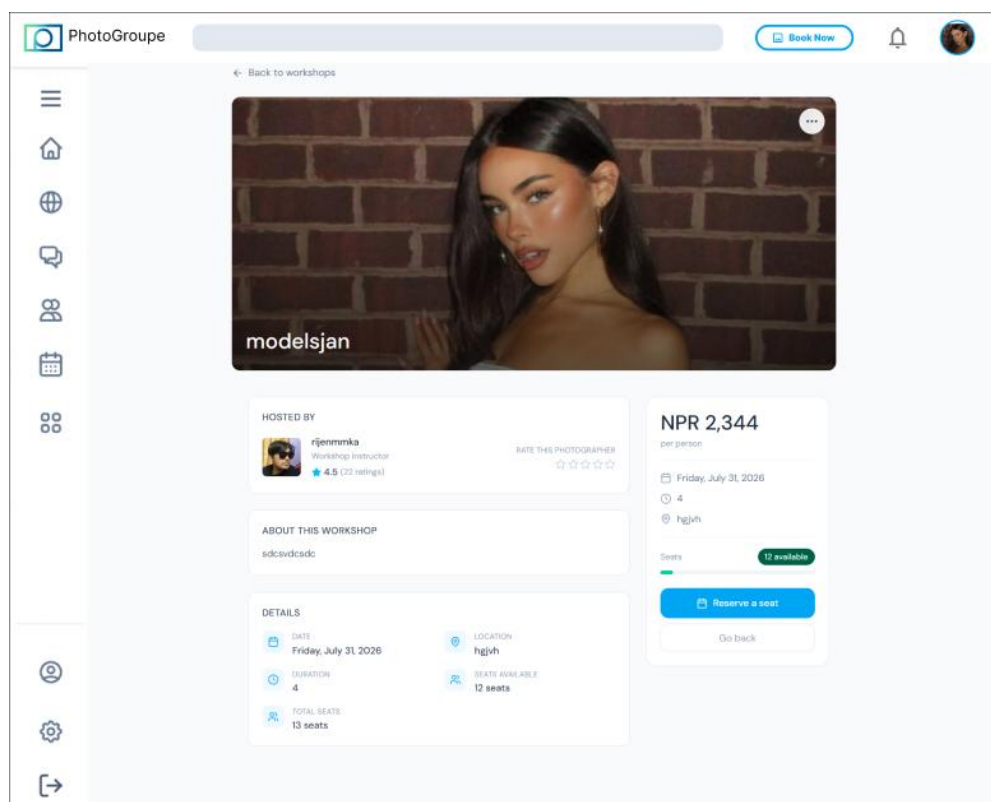


Figure 3.10.4.2: UI Wireframe – Workshop detail viewing and reserve seats page

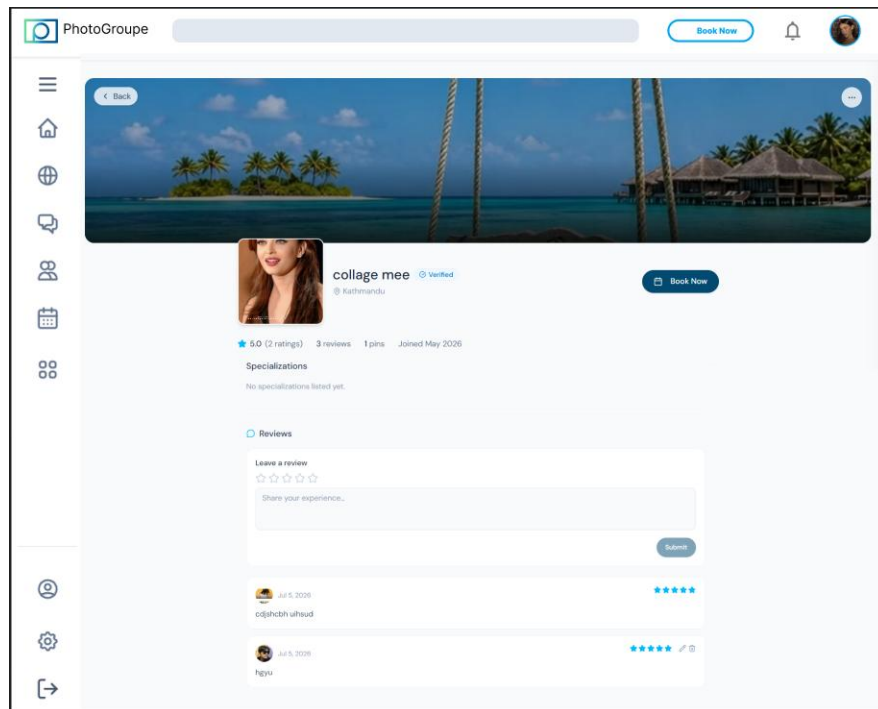


Figure 3.10.4.3: UI Wireframe – Photographer Portfolio

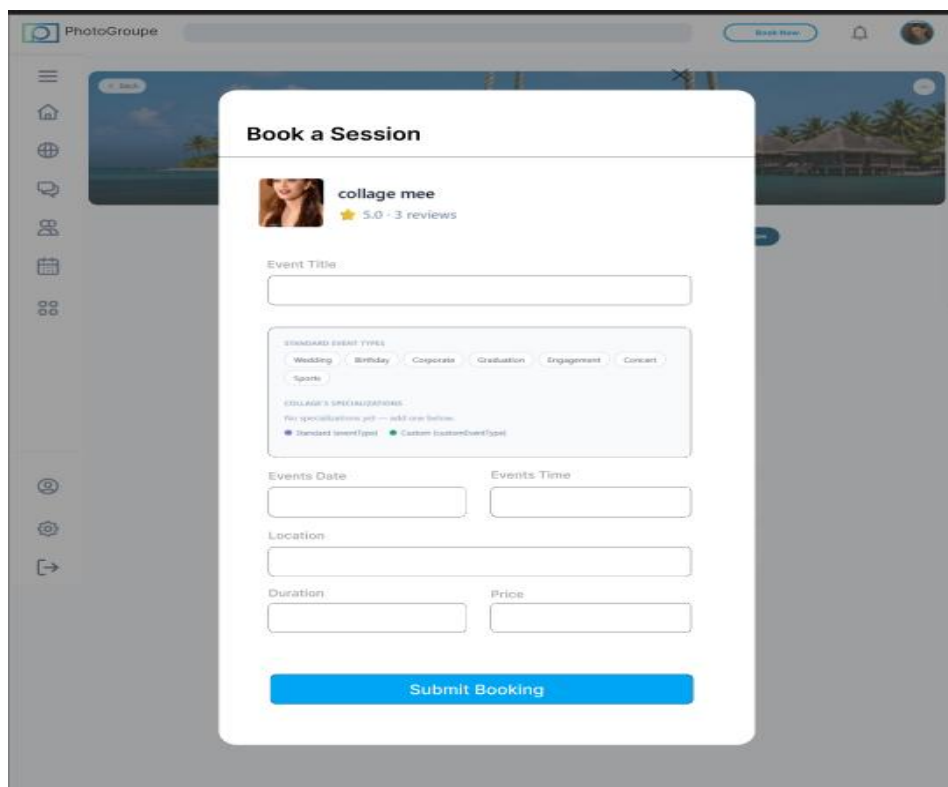


Figure 3.10.4.4: UI Wireframe - Booking Registration

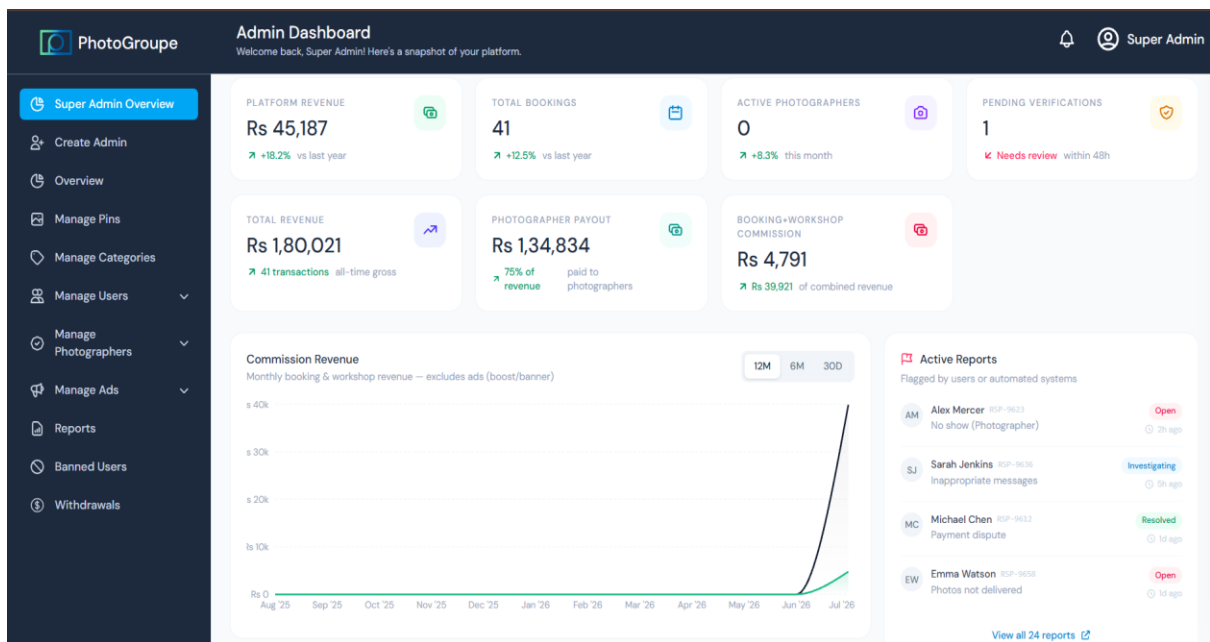


Figure 3.10.4.5: UI Wireframe - Admin Dashboard Overview

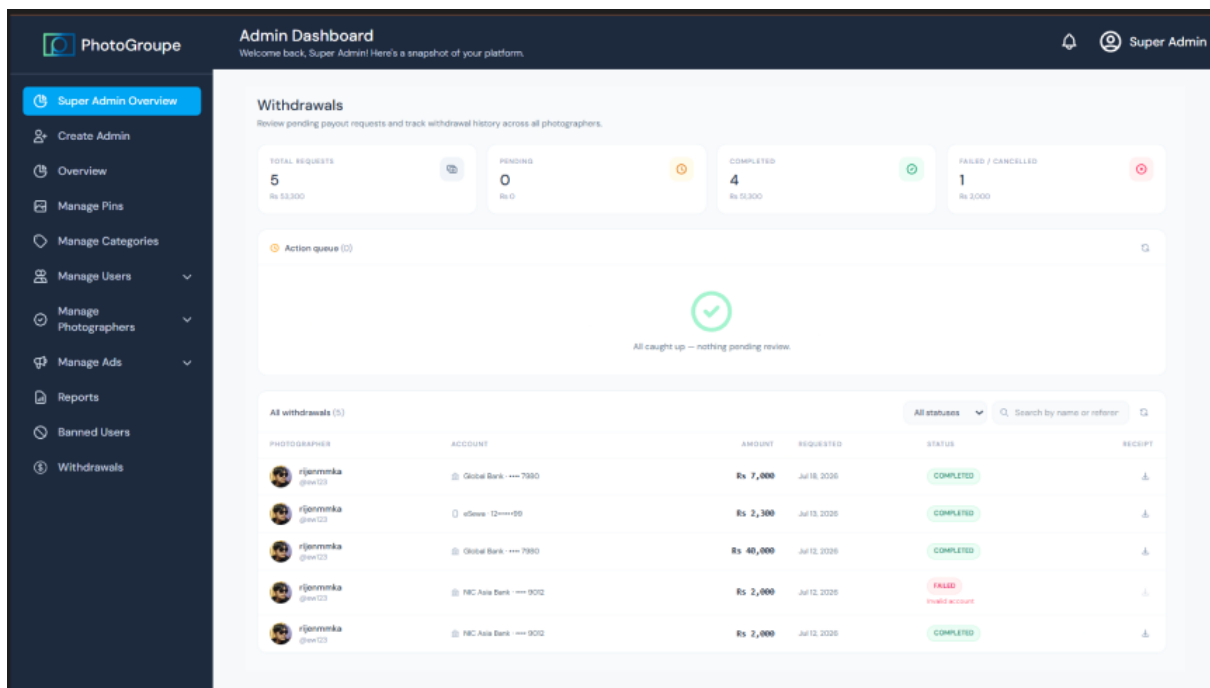


Figure 3.10.4.6: UI Wireframe - Admin Transactions Panel

CHAPTER 4: SYSTEM IMPLEMENTATION

4.1 Introduction

This chapter presents the practical implementation of the design discussed in Chapter 3. PhotoGroupe was built as a full-stack web application using React on the frontend and Spring Boot on the backend, with JPA/Hibernate for data persistence, Cloudinary for media storage, and eSewa for payments. It covers the technology stack and development setup, how core modules (pins, albums, bookings, bidding, workshops, chat, admin panel) were implemented, the runtime security mechanisms (JWT, OAuth2, role-based access), the REST API conventions used, version control practices, system requirements, and the current deployment status all based directly on the actual PhotoGroupe codebase.

4.2 Technologies Used

Table 4.2.1: Technology Stack

Layer	Technology	Version	Purpose
Framework	Spring Boot	4.0.4	REST API, dependency injection, application server
Language	Java	17	Backend implementation language
Security	Spring Security + JWT	0.11.5	Authentication filter chain, JWT issuing/validation
OAuth	Spring OAuth2 Client	Boot-managed	Social login (OAuth2)
Persistence	Spring Data JPA / Hibernate	Boot-managed	ORM over MySQL
Database	MySQL (mysql-connector-j)	Boot-managed	Primary relational data store
Real-time	Spring WebSocket	Boot-managed	Chat and live notification channel
Email	Spring Boot Starter Mail	Boot-managed	Transactional and notification emails
PDF generation	Apache PDFBox	3.0.2	PDF document generation (reports, receipts)
Media storage	Cloudinary (cloudinary-http5)	2.0.0	Image/file hosting and transformation
Config management	dotenv-java	3.2.0	Environment variable loading

Boilerplate reduction	Lombok	Boot-managed	Getters/setters/constructors via annotations
Validation	Spring Boot Starter Validation	Boot-managed	Bean validation on request payloads
Build tool	Maven	—	Dependency management and build
Framework	React	19.1.1	SPA UI
Build tool	Vite	7.1.2	Dev server and bundler
Styling	Tailwind CSS	4.1.12	Utility-first CSS
UI component library	Ant Design (antd)	5.27.0	Prebuilt UI components
Utility styling	class-variance-authority, clsx, tailwind-merge, Radix Slot	latest	Component variant and class-merging utilities
Routing	React Router	7.8.1	Client-side routing
HTTP client	Axios	1.11.0	REST calls to the backend
Real-time (client)	@stomp/stompjs + sockjs-client	7.3.0 / 1.6.1	WebSocket/STOMP client for chat and notifications
Charts	Recharts	3.8.1	Admin/photographer analytics
Animation	Framer Motion	12.40.0	UI transitions and animations
Icons	Lucide React, react-icons	latest	Icon sets
Image cropping	react-image-crop	11.0.10	Profile/pin image cropping
Bot protection	react-google-recaptcha	3.1.0	Form spam protection
Notifications (UI)	react-toastify	11.0.5	Toast alerts
Date handling	date-fns	4.2.1	Date formatting and manipulation
Dev/mock tooling	json-server, Express, Multer, CORS	latest	Local API mocking and dev tooling
Payments	eSewa (custom integration)	—	Local Nepali payment gateway

4.3 Development Environment

- **IDE:** IntelliJ IDEA/Eclipse for the Spring Boot backend and VS Code for the React frontend.
- **Backend:** Java 17 (LTS) with Maven and Spring Boot.
- **Frontend:** React with Vite, using Node.js and npm.
- **Database:** MySQL 8 with Hibernate (ddl-auto=update) for automatic schema updates.
- **API Testing:** Postman collections used to test REST APIs before frontend integration.
- **Version Control:** Git and GitHub with separate repositories for backend and frontend.
- **Local Setup:** Backend runs on localhost:8080 and frontend on localhost:5173, with CORS configured for local and LAN access.
- **Payment Testing:** Sandbox environment of eSewa (EPay v2) used during development for booking and workshop payments.

4.4 Module-wise Implementation

- **Auth Module:** Password + Google OAuth2 login, JWT access/refresh tokens, OTP for forgot-password, role-based access control
- **Profile:** Profile CRUD, follow/block, photographer specializations
- **Pin/Content:** Pin CRUD, categories, comments, likes
- **Album:** Album CRUD, ZIP download, visibility control
- **Booking:** Booking flow, package negotiation, penalty enforcement
- **Bidding:** Event requests + photographer bids
- **Workshop:** Workshop creation, seat management, registrations
- **Payment:** eSewa integration for bookings & workshops
- **Payout:** Commission-based payouts, escrow release
- **Chat:** Real-time WebSocket messaging, inline booking negotiation

- Notification: Real-time alerts for likes, bookings, bids
- Admin: Reports, monetization, workshop/booking oversight
- Rating: Bayesian-weighted photographer ranking
- Sharing: Generic share modal for pins, albums, workshops.

4.5 Security Implementation

4.5.1 User Authentication

When a user logs in, PhotoGroupe checks their email and password against what was stored at signup. If they match, the server issues a signed JWT access token that acts like a temporary ID card the frontend attaches this token to every subsequent request instead of resending the password. A separate, longer-lived refresh token lets the app quietly renew access without forcing the user to log in again. Beyond password login, PhotoGroupe also supports signing in directly with a Google account via OAuth2, and a one-time code (OTP) is used specifically for the forgot-password flow, letting a user securely reset their password without needing to remember the old one.

4.5.2 Authorization and Role Management

PhotoGroupe recognizes four account types: regular users, photographers, admins, and super admins. Every request is checked against the caller's role before it's allowed through endpoints under `/api/admin/**` are restricted to ADMIN and SUPER_ADMIN, while `/api/photographer/**` is restricted to PHOTOGRAPHER, ADMIN, and SUPER_ADMIN. This check happens fresh on every single request rather than once at login, so a regular user account can never reach admin or photographer-only endpoints no matter what it tries, and a role change takes effect immediately rather than only after a fresh login.

4.5.3 Password Hashing

Passwords are never stored in a form anyone could read. When an account is created, the password is passed through a one-way hashing process that turns it into a fixed, unreadable string not even the platform's own database administrator can recover the original from it. At login, the entered password is hashed the same way and compared against the stored version; a match grants access.

4.5.4 Session Management

The server doesn't keep a running memory of "who's logged in." Session creation policy is set to stateless, so identity is proven fresh on every request using the JWT itself rather than server-side session storage. This keeps the system lightweight and easy to scale any server instance handling the app can process any request without needing to know what happened on a previous one and it makes logging out as simple as the app discarding its saved token.

4.5.5 Input Validation

Before sensitive actions are carried out, the system checks that the request actually makes sense for example, verifying seat availability before a workshop registration, checking payout account details before releasing funds, or validating booking package terms during negotiation. Public endpoints (search, browsing pins/categories, authentication routes, eSewa callbacks) are explicitly whitelisted, while everything else defaults to requiring authentication, so no endpoint is accidentally left open.

4.5.6 Data Protection and Security Measures

Sensitive configuration API keys, database credentials, JWT signing secrets is kept out of the codebase and supplied only at runtime through environment variables. Cross-origin requests are restricted through an explicit CORS configuration rather than left open to any site on the internet, and preflight (OPTIONS) requests are handled separately to support this safely. CSRF protection is disabled in favor of stateless JWT-based authentication, which is standard practice for token-secured REST APIs that don't rely on cookies for session identity.

4.6 API Design and Implementation

The API follows REST conventions over JSON: resource-oriented URLs (/api/products, /api/membership/levels), HTTP verbs mapped to CRUD (GET/POST/PUT/PATCH/DELETE), and controller methods kept thin, delegating to services. A representative slice of the 100+ endpoints across the 28 controllers:

Table 4.6.1: Sample API Endpoints

Endpoint	Method	Purpose
/auth/login	POST	Password-based login, returns JWT
/auth/forgot-password / verify-otp / reset-password	POST	OTP-based password reset

/api/users/{id}	GET	Public user/photographer profile
/api/users/pins	GET / POST	List pins / upload a new pin
/api/users/albums/{id}/download	GET	Download album pins as a ZIP
/api/users/categories	GET	List browsable pin categories
/api/users/events	POST	Client posts an event request for bidding
/api/users/bids/{id}/accept	PATCH	Client accepts a photographer's bid
/api/users/bookings	POST	Create a booking with a photographer
/api/users/bookings/{id}/complete	PATCH	Complete booking, triggers payout release
/api/users/workshops/{id}/register	POST	Register as a workshop participant
/api/users/esewa/success	GET	eSewa payment verification callback
/api/users/payouts	GET	View payout history
/api/users/follows/{username}	POST / DELETE	Follow / unfollow a user
/api/users/messages	POST	Send a chat message
/api/users/notification	GET	List notifications for the user
/api/users/reports	POST	Submit a moderation report
/api/admin/stats/overview	GET	Platform-wide dashboard statistics
/api/admin/reports/{id}/resolve	PATCH	Resolve a reported item

4.7 Version Control and Project Management

4.7.1 Git/GitHub Workflow

The project is hosted on GitHub as two repositories: PhotoGroupe-Frontend and Final-Year-Project- (backend). As of this writing: 8 commits on the backend and 24 commits on the frontend, tracked from local Git across development.

4.7.2 Branching Strategy

Development has been trunk-based on a single main/master branch with direct commits, appropriate for a solo-developer final year project where coordinating pull requests across contributors isn't a real constraint. This is a deliberate simplification rather than an oversight a multi-branch Git Flow model would add process overhead without a second contributor to isolate work from.

4.7.3 Issue Tracking (GitHub)

No formal issue tracker (e.g. GitHub Issues) has been used to date; task tracking has been informal, matching an iterative-incremental delivery approach each commit corresponds to one completed feature slice (e.g. admin panel modules, booking payment flow, dark/light theme rollout) rather than a ticket. Adopting GitHub Issues would be a reasonable process improvement if the project moves beyond a solo capstone.

4.8 Resources Required

4.8.1 Software Requirements

Table 4.8.1: Software Requirements

Category	Requirement
OS	Windows / macOS / Linux (cross-platform)
JDK	17+
Node.js	18+ (for Vite 8 / React 19)
Database	MySQL 8
Build tools	Maven (backend), npm/Vite (frontend)
IDE	IntelliJ IDEA / VS Code
API testing	Postman
Version control	Git + GitHub account

Accounts	Cloudinary, Google Cloud (OAuth2 client), eSewa merchant (sandbox)
----------	--

4.8.2 Hardware Requirements

Table 4.8.2: Hardware Requirements

Environment	Minimum spec
Development machine	Dual-core CPU, 8 GB RAM, 10 GB free disk (IDE + JDK + Node + MySQL + browser)
Production (early scale)	1–2 vCPU, 2–4 GB RAM VPS — sufficient for a single Spring Boot instance + MySQL at low transaction volume

4.9 Deployment Details

4.9.1 Hosting Environment

The application currently runs in local development only (localhost:8080 / localhost:5173); it has not yet been deployed to a production host. A low-cost VPS is the planned target for eventual deployment.

4.9.2 Server Configuration

Locally: MySQL 8 on its default port, the Spring Boot app on 8080, and the Vite dev server on 5173, with the frontend base URL and the eSewa callback URLs in application.properties pointed at localhost these would need to change to the production domain at deploy time.

4.10 Project Timeline (Gantt Chart)

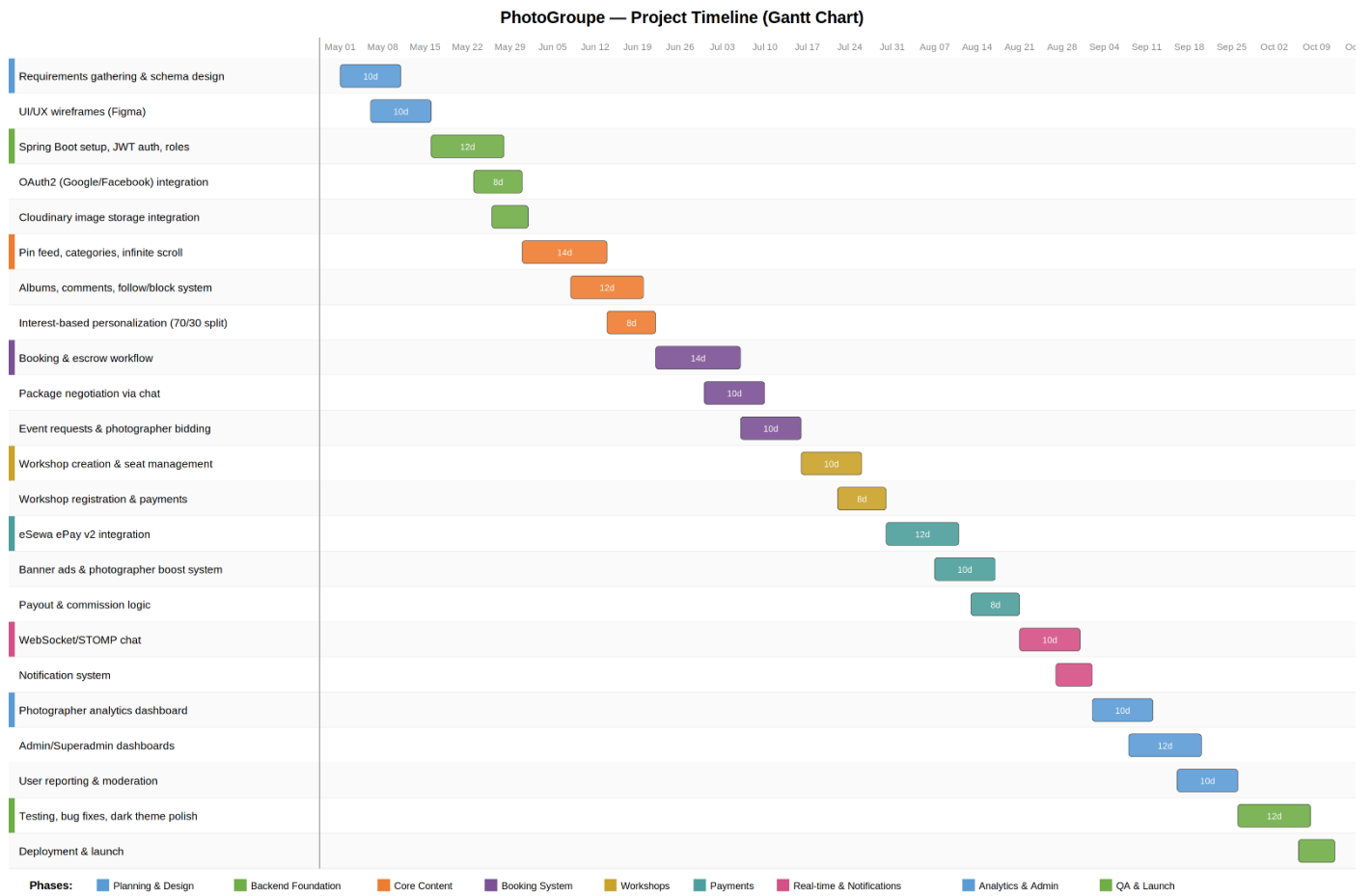


Figure 4.9.2.1: Project Timeline - Gantt Chart

CHAPTER 5: TESTING AND EVALUATION

5.1 Testing Strategy

PhotoGroupe was tested at three levels unit, integration, and manual system — chosen to match where bugs are actually costly: a silent mistake in payment, escrow, or moderation logic is far more damaging than one in a cosmetic UI detail, so automated coverage was concentrated there first.

5.1.1 Unit Testing

Backend business logic is tested in isolation using JUnit 5 and Mockito, with repositories mocked out so a test exercises only the service's own logic. like for e.g.: `BookingServiceTest` (11 tests) verifies booking creation validation (missing event type, date conflicts, restricted clients, non-photographer targets), cancellation rules (ownership, already-completed bookings), and escrow payment release, including correct calculation of the 12% platform commission. All 11 unit tests passed, covering the payment, booking, and moderation logic most critical to the platform's correctness

5.1.2 Integration Testing

Higher up the stack, `@SpringBootTest` exercises the real Spring context actual security configuration, JPA entity mappings, and controller-to-service wiring against an in-memory H2 database, so these tests catch integration problems that pure unit tests can't see. `BookingIntegrationTest` is the main guard here it locks in the full create-and-persist flow, so a future change to the service or repository layer can't silently break booking creation or its downstream notification without a test failing.

5.1.3 System Testing

The full booking lifecycle from a client submitting a request through to payment release spans multiple roles, real-time chat, and escrow state transitions that are difficult to fully simulate, so it was manually walked through end-to-end against the running frontend and backend together. This covered: a client submitting a booking request to a photographer, the photographer confirming or rejecting it, package negotiation via chat (send, counter-offer, accept), payment moving into escrow, the photographer marking the session complete, and the client releasing payment checking at each step that booking status, escrow status, and both parties' notifications updated correctly and consistently across the client and photographer dashboards.

5.2 Test Cases and Results

Testing was carried out across four levels unit, integration, system, and user acceptance covering the major modules of PatronNP: authentication, payments, payouts, membership, shop, content, follow, moderation, and admin. A representative sample is shown below; further test cases exist covering additional scenarios and edge cases across each module.

Table 5.2.1: Sample Test Cases and Results

Test Case ID	Module	Test Scenario	Expected Result	Status
UT-001	Booking	Create booking targeting a non-photographer user	Throws exception "User is not a photographer"	Pass
UT-013	Report	Report with reason = OTHER and no message	Throws exception message required	Pass
IT-001	Booking	Full booking creation persisted via real service + repository (Spring context, H2)	Booking + notification rows committed with valid foreign keys	Pass
IT-002	Album	Photographer creates an album with title, description, and visibility via real controller + service + repository	Album row persisted to H2 with correct title and owner (user_id foreign key)	pass
IT-003	Album	Create an album without a cover image (optional file field)	Album still created successfully, coverImageUrl remains null	Pass
ST-	Payment	eSewa sandbox	Payment callback	Pass

006		payment round trip for a booking	verifies transaction, booking's payment status updates to PAID/escrow HELD	
ST-008	Guest Access	Unauthenticated user browses Explore feed	Pins load normally without triggering an auth error or redirect	pass
UAT-001	Onboarding	First-time user completes onboarding unaided	—	Not Executed

5.3 Performance Evaluation

5.3.1 Response Time

This checks how fast the app replies. I tested it by asking the server the same questions 30 times each:

- Global Search: about 29 to 48 milliseconds on average, with one outlier spiking to 428ms
- Loading a single booking by ID: about 10 to 25 milliseconds
- Loading a client's bookings list: about 9 to 31 milliseconds

A millisecond is one-thousandth of a second, so most of these still feel instant to a user.

5.3.2 Scalability

The app doesn't need to "remember" who's logged in, since each request carries its own proof of identity (a token called a JWT). This means the app can run on several computers at once instead of just one, sharing the workload evenly. The database is the one part that can't be copied this way, so it would be the first thing to upgrade if usage grew a lot.

5.3.3 Resource Utilization (CPU, Memory, Storage)

While testing, the app's Java process (PID 30908, confirmed as the one bound to port 8080) used about 500MB of memory. CPU usage under heavy traffic wasn't tested yet; that would

be a good next step before going live. Storage usage (database file size, upload directories) hasn't been measured yet either.

5.4 Results and Discussion

The project succeeded at its main goals: clients can book photographers, negotiate packages through real-time chat, and pay safely into escrow that's only released once the session is confirmed complete. The trickiest part was getting the escrow and commission logic right the app calculates the 12% platform fee and only releases payment after the client confirms the session was completed, instead of trusting either side's word.

5.5 Comparison with Objectives

- Secure Login: Achieved. Passwords are stored hashed, with JWT-based authentication.
- Booking System: Achieved. Clients can request, cancel, confirm, and reject bookings with proper validation (no double-booking, no non-photographer targets).
- Escrow Payments: Achieved. Payments are held in escrow and only released after client confirmation, with automatic 12% commission calculation.
- Real-Time Chat & Negotiation: Achieved. Clients and photographers can message, counter-offer, and accept packages in real time.
- Moderation & Reporting: Achieved. Reports are validated (e.g. reason required for "Other") and reviewed by admins.
- Going Live: Not Achieved. The app currently only runs on the developer's own computer and hasn't been deployed yet.

5.6 Challenges Faced

- Getting escrow release timing right so payment only moves after genuine session completion
- Calculating the 12% platform commission accurately across different booking scenarios
- Keeping booking/escrow status in sync in real time across both client and photographer dashboards

- Simulating the full multi-role booking lifecycle (chat + escrow + status transitions) for manual testing

CHAPTER 6: CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Summary

PhotoGroupe is an image-sharing and photographer booking platform built for the Nepali market, letting users share and discover images the way they would on Pinterest, while also browsing verified photographers, negotiating packages over real-time chat, and booking them through an escrow-protected payment flow settled via eSewa. Built solo using an Agile, iterative-incremental methodology — each module (auth → pins/categories → albums → booking → bidding → workshops → chat → payments → admin) delivered as its own working increment — the platform covers authentication and role-based access, portfolio and album management, booking and escrow payments, an event-request bidding system, workshops, real-time chat, notifications, and an admin panel, with the highest-risk logic (booking rules, escrow release, commission calculation) backed by automated unit and integration tests.

6.2 Limitations

- Photographer verification is reviewed manually by an admin, since no automated identity-verification service is integrated.
- The system runs on a single MySQL instance with no load balancing or backup automation in place.
- It has not been deployed publicly; it currently runs only on the developer's own computer (localhost).
- Payment integration is currently limited to eSewa, with Khalti support planned but not yet implemented.
- Automated test coverage is concentrated on booking, payment, and moderation logic; other modules rely more on manual system testing. .

6.3 Future Enhancements

- Complete Khalti integration alongside eSewa for more local payment options.
- Automate photographer verification to reduce manual admin review.
- Deploy the platform to a live VPS for public access.
- Load-test under heavy traffic and scale the database if needed.

- Introduce AI-powered pose recommendations for photographers and clients, based on photoshoot category.
- Add smarter AI-based photographer-client matching and personalized recommendations.
- Extend automated image tagging using AI.
- Test the app under heavy simulated traffic and upgrade the database setup if needed.

6.4 Conclusion

PhotoGroupe demonstrates that image sharing and professional photographer booking can be meaningfully combined into a single platform, rather than forcing photographers to stitch together separate tools for exposure, client communication, and payment. By pairing a familiar Pinterest-style sharing experience with booking, escrow payments, and bidding none of which the reviewed competitor platforms offer PhotoGroupe gives photographers a direct path from visibility to paid work, and gives clients a safer, more structured way to find and hire them. The clearest next step before a real launch is production deployment, alongside load testing to confirm the platform holds up under real-world concurrent traffic.

REFERENCES

- Aftershoot Team. (2026, February 9). *Social media for photographers: What to post and when*. Aftershoot. <https://aftershoot.com/blog/social-media-for-photographers/>
- Aftershoot Team. (2026). *Best social media for photographers*. ShootProof. <https://www.shootproof.com/blog/best-social-media-for-photographers/>
- Almeida, F., & Monteiro, F. (2017). Web development frameworks and user experience. *International Journal of Computer Applications*, 162(4), 15–22.
- Anderson, T. (2020). Digital platforms and creator economies. *Journal of Digital Media Studies*, 12(3), 45–58.
- Colombo, M. G., Franzoni, C., & Rossi-Lamastra, C. (2015). Internal social capital and the attraction of early contributions in crowdfunding. *Entrepreneurship Theory and Practice*, 39(1), 75–100.
- Escudero, L., Anglin, A., Allison, T., & Wolfe, J. (2025). Sustainable monetization strategies for digital platforms. *Journal of Online Business*, 14(2), 88–104.
- Fiveable Content Team. (2025). *Social media and photo sharing platforms*. Fiveable. <https://fiveable.me/history-of-photography/unit-10/social-media-photo-sharing-platforms/study-guide/jrq0cWPJ7zkHWEF3>
- Fstoppers. (2023, June 16). *The pressures of social media on photographers*. <https://fstoppers.com/social-media/pressures-social-media-photographers-633956>
- Instagram Help. (n.d.). *About Instagram*. <https://help.instagram.com/424737657584573/>
- Johnson, R., & Miller, S. (2020). Social media marketing for photographers. *International Journal of Media Studies*, 8(2), 115–128.
- Konstan, J. A., & Riedl, J. (2012). Recommender systems: From algorithms to user experience. *User Modeling and User-Adapted Interaction*, 22(1–2), 101–123.
- Kumar, P., & Patel, A. (2022). The role of social media platforms in visual content sharing. *Journal of Online Communication*, 10(1), 72–85.
- Octal Software. (2025). *Top 15 image sharing apps like Pinterest in 2025*. <https://www.octalsoftware.com/blog/apps-like-pinterest>

PicDrop. (n.d.). *Social media for photographers: A guide and usage comparison*. PicDrop.
<https://www.picdrop.com/web/articles/social-media-platforms-for-photographers>

Pinterest Help. (n.d.). *All about Pinterest*. <https://help.pinterest.com/en/guide/all-about-pinterest>

Smith, L. (2021). The evolution of social media platforms in digital photography. *Media and Technology Review*, 14(2), 88–101.

The Knowledge Academy. (2026). *Pinterest alternatives: Best sites like Pinterest*.
<https://www.theknowledgeacademy.com/blog/pinterest-alternatives/>

WafaTech. (2025). *Scalable web application frameworks and global deployment*. WafaTech White Paper.

Williams, M. (2026, January 5). *The best photo sharing sites in 2026*. PetaPixel.
<https://petapixel.com/best-photo-sharing-sites/>

Wolinski, E. (2025, June 10). *The best photo storage and sharing sites in 2025*. Tom's Guide. <https://www.tomsguide.com/best-picks/best-photography-sites>

